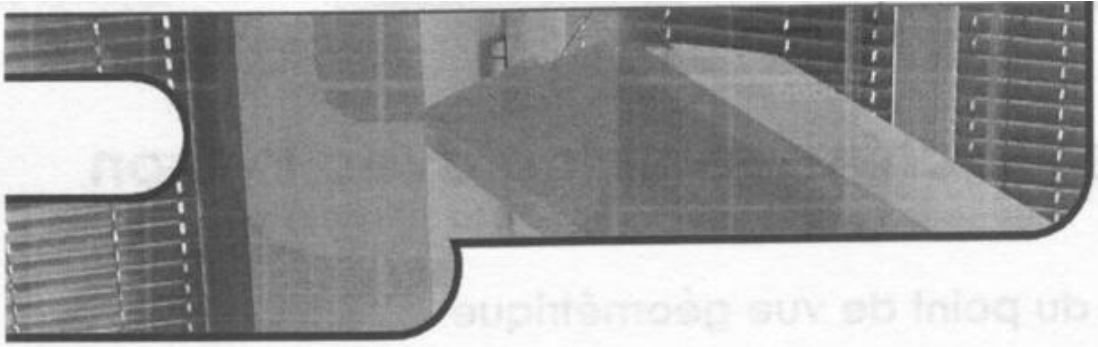


Chapitre 4-5
Support Vector Machine

1. Objectif du chapitre

L'algorithme SVM fut introduit par Vapnik en 1995 et il est basé sur la théorie de Vapnik-Chervonenkis. Théoriquement, les SVM sont une généralisation des modèles de classification linéaires. Les SVM ont été appliqués avec beaucoup de succès dans divers domaines et spécialement dans le domaine de la reconnaissance de formes à partir d'images et le domaine de la bio-informatique.

Dans la littérature francophone, l'algorithme SVM est le plus souvent nommé «algorithme séparateur à vaste marge». En effet, les algorithmes SVM se distinguent de la plupart des autres algorithmes par la maximisation de la marge qui sépare deux classes de données. Ce chapitre tente d'expliquer les SVM d'un point de vue intuitif, mais aussi analytique, tout en mettant l'accent sur cet aspect de maximisation de la marge séparatrice.

À la fin de ce chapitre, le lecteur aura abordé :

- une compréhension intuitive de l'algorithme SVM,
- les SVM du point de vue théorique et analytique,
- la notion de fonction noyau,
- le théorème de Mercer,
- l'application de l'algorithme SVM pour la réalisation d'un modèle de détection de fraudes de cartes de crédit à partir d'un jeu de données réel contenant des données relatives à des transactions bancaires.

2. Le SVM du point de vue géométrique

Les SVM sont des algorithmes supervisés qui, au début, ont été développés pour résoudre des problèmes de classification binaire. Ils ont été généralisés pour prédire des variables quantitatives.

Dans le cas de la classification binaire, les SVM recherchent un hyperplan discriminant de marge optimale entre deux classes. Pour comprendre l'idée véhiculée par cet algorithme, cette section donne une explication intuitive basée sur des illustrations.

Dans la figure ci-dessous, deux ensembles sont représentés sur quatre graphiques à deux dimensions : le premier est composé de neuf carrés vides et le deuxième ensemble est composé de neuf cercles vides.

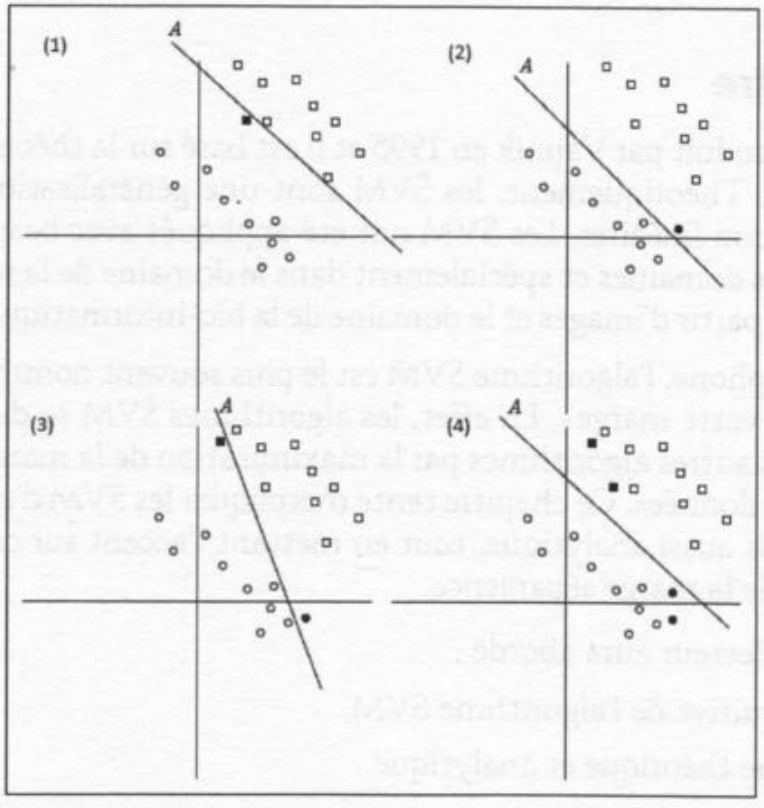


Figure11-1 : exemples de modèles linéaires

© Editions ENI - All rights reserved

Supposons que ces carrés et ces cercles vides forment la base de données d'apprentissage et que les carrés et les cercles noirs (pleins) sont les données futures, des données encore inconnues au moment de la phase d'apprentissage.

Sans prendre en compte ces données futures, le modèle représenté par la droite *(A)* dans chacun des quatre repères sépare toutes les données d'apprentissage de façon correcte. Cependant, des différences existent entre ces quatre modèles. En effet, bien qu'ils soient égaux vis-à-vis des données d'apprentissage, ils n'ont pas le même pouvoir de généralisation sur les données futures. Ainsi :

- Dans le repère (1), le modèle considère à tort que le point carré noir est un cercle puisqu'il est du côté gauche de la droite *(A)*.
- Dans le repère (2), le modèle considère à tort que le cercle plein est un carré puisqu'il est du côté droit de la droite *(A)*.
- Dans le repère (3), le carré noir et le cercle noir sont mal classés puisqu'ils sont du mauvais côté de la droite *(A)*.
- Dans le repère (4), toutes les données futures, cercles et carrés pleins, sont correctement classées.

Parmi ces quatre modèles, celui du repère (4) est le plus performant puisque non seulement il classe tous les exemples d'apprentissage correctement, mais en plus il classe correctement les exemples des données futures.

De façon intuitive, le modèle représenté par la droite *(A)* du repère (4) présente une meilleure capacité de généralisation, car cette droite est positionnée de façon à séparer correctement les données d'apprentissage, les carrés et les cercles vides. De plus, cette droite *(A)* est le plus loin possible de toutes ces données.

C'est justement cette idée impliquant de trouver un hyperplan le plus loin possible des données séparées qui est exploitée par le SVM. Les sections suivantes détaillent comment le SVM trouve un tel hyperplan.

La démarche de SVM

La figure 11-2 ci-après explique comment le SVM procède pour classer les données futures ou les données de production. Cette figure reprend le même exemple que celui du repère (4) de la figure précédente. Par exemple, pour trouver la classe d'appartenance du point P_1, le SVM effectue la projection de ce point sur un vecteur $\vec{w}$ normal à la droite *(A)*. En fonction du résultat de cette projection, le SVM prend la décision sur la classe d'appartenance de *P*1. Cette décision est prise en se basant sur les deux règles suivantes :

Si $\overrightarrow{p_1}.\vec{w} \geq B$, alors P_1 est un carré.

Si $\overrightarrow{p_1}.\vec{w} \leq B$, alors P_1 est un cercle.

La valeur de la constante *B* et le vecteur $\vec{w}$ sont calculés sur les données d'apprentissage. La section suivante montre comment ces deux éléments sont calculés.

On note que si $\overrightarrow{p_1}.\vec{w} = B$, alors le point sera considéré simultanément comme étant un cercle et un carré !

La projection de P_1 se situe au-dessus de la droite *(A)*, donc le point *P*1 sera classé dans la classe des carrés.

Cet exemple illustre bien l'importance de la position de la droite séparatrice *(A)*. En effet, cette droite peut être décalée légèrement en diagonale vers le haut ou en diagonale vers le bas, tout en gardant les données d'apprentissage bien séparées. En revanche, le point P_1 peut être classé comme un carré ou comme un cercle, en fonction de la position de cette droite séparatrice.

En d'autres termes, le SVM ne se contente pas de trouver un hyperplan qui sépare correctement les données d'apprentissage, il cherche en plus l'hyperplan qui s'apprête le mieux à la généralisation.

Le but du SVM est donc de trouver les éléments $\vec{w}$ et *C* en utilisant les données d'apprentissage, puis appliquer les deux règles de décision précédemment citées pour classer les données futures.

© Editions ENI - All rights reserved

Plusieurs algorithmes, autres que les algorithmes SVM, recherchent un hyperplan séparant correctement les données de la base d'apprentissage. Les SVM, comme évoqué lors des exemples précédents, recherchent la position optimale de cet hyperplan.

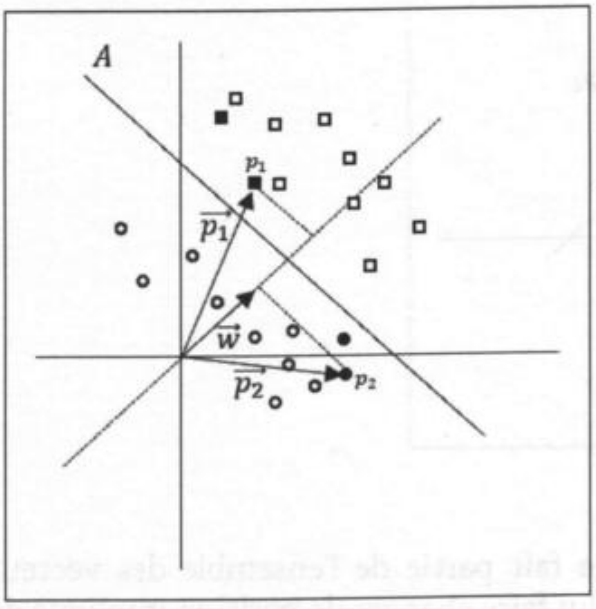


Figure11-2 : classification des données de production par le SVM

Cette position optimale peut être obtenue en maximisant les deux distances d_1 et d_2 (voir figure 11-3 ci-dessous). En d'autres termes, le SVM cherche à maximiser la marge qui sépare la classe des carrés de la classe des cercles. Dans l'exemple de cette figure, cette marge optimale entre les deux classes peut être représentée par les deux droites S_1 et S_2 et elle est obtenue en se basant sur les points p_1, p_2 et p_3. En effet, la droite *(A)* est placée à l'endroit où elle est la plus éloignée de p_1 et p_2 d'un côté, et p_3 de l'autre côté. Le SVM a choisi les points p_1, p_2 et p_3 pour déterminer la position de la droite *(A)*, car ces trois points représentent les frontières entre la classe des cercles et la classe des carrés. Ces points sont appelés les vecteurs supports, d'où le nom de Support Vector Machine !

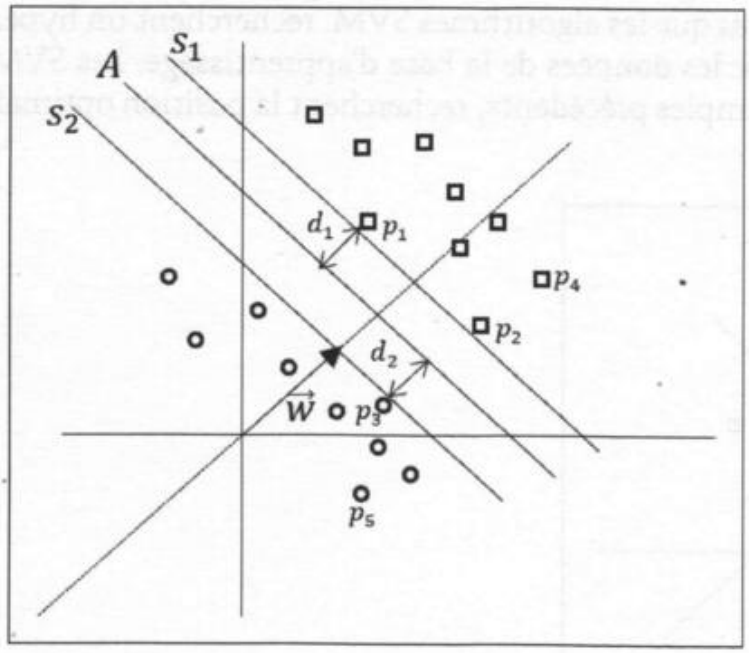


Figure 11-3 : séparateur à vaste marge

Un point de la base d'apprentissage fait partie de l'ensemble des vecteurs supports si et seulement si le fait de lui faire changer de position implique des modifications de la marge maximum séparant les deux classes.

Par exemple, dans la figure 11-3, supprimer le point p_3 a pour conséquence de déplacer la droite S_2 vers la gauche et ainsi d'élargir encore plus la marge qui sépare les deux classes. p_3 est donc un vecteur support. Pour la même raison, les points p_1 et p_2 sont également des vecteurs supports. En revanche, les deux points p_4 et p_5 n'en sont pas puisque le fait de les supprimer ne changera pas l'étendue de la marge qui sépare les deux classes.

3. Le SVM du point de vue analytique

L'objectif de l'algorithme SVM est de trouver l'hyperplan qui sépare en deux classes avec la marge maximale les données de la base d'apprentissage. Cette section montre la démarche analytique pour atteindre cet objectif.

© Editions ENI - All rights reserved

Supposons que les deux classes à séparer sont la classe des exemples positifs et celle des exemples négatifs. Nous avons vu qu'une fois l'hyperplan séparateur trouvé, pour déterminer la classe d'appartenance d'un point x_i, le SVM utilise une constante C et le vecteur $\vec{w}$, qui est un vecteur normal à l'hyperplan, comme suit :

Si $\vec{w}.\vec{x}_i \geq B$, alors x_i est positif.

Si $\vec{w}.\vec{x}_i \leq B$, alors x_i est négatif.

Si nous remplaçons B par $-b$, nous obtenons :

$\vec{w}.\vec{x}_i + b \geq 0$, pour les futurs points x_i positifs

$\vec{w}.\vec{x}_i + b \leq 0$, pour les futurs points x_i négatifs

Avec les deux inéquations précédentes, déterminer la classe d'un point x_i revient à étudier le signe de la quantité $\vec{w}.\vec{x}_i + b$. S'il est positif, alors x_i appartient à la classe des points positifs, sinon il appartient à la classe des points négatifs. En résumé, la classe d'un point x_i est déterminée par :

$Sign[\vec{w}.\vec{x}_i + b]$ (1)

L'équation (1) constitue donc la règle de décision que le SVM utilise pour la prédiction de la classe d'appartenance des futurs points. Cette règle est basée sur les deux inconnues $\vec{w}$ et b. L'objectif de la suite de cette section est d'expliquer les étapes analytiques nécessaires pour trouver les valeurs de ces deux inconnues.

Pour les points de la base d'apprentissage, la recherche des deux paramètres $\vec{w}$ et b peut être basée sur les deux contraintes suivantes :

$\vec{w}.\vec{x}_i + b \geq 1$, pour les points de la base d'apprentissage positifs (2)

$\vec{w}.\vec{x}_i + b \leq -1$ pour les points de la base d'apprentissage négatifs (3)

Les deux dernières contraintes permettent de restreindre la recherche de $\vec{w}$ et b de façon à ce que les données d'apprentissage positives soient éloignées des données d'apprentissages négatives et créer une frontière entre les deux classes.

Pour unifier ces deux contraintes, une nouvelle variable y_i est introduite, définie comme suit :

$y_i = 1$ pour les x_i positifs

$y_i = -1$ pour les x_i négatifs

Si on multiplie les deux équations (2) et (3) par y_i, on obtient :

Pour les $\vec{x}_i$ positifs et $y_i = 1$, $y_i(\vec{w}.\vec{x}_i + b) \geq 1$ (4)

Pour les $\vec{x}_i$ négatifs et $y_i = -1$, $y_i(\vec{w}.\vec{x}_i + b) \leq -1$ (5)

On multiplie l'inéquation (5) par -1 et on obtient : $y_i(\vec{w}.\vec{x}_i + b) \leq -1$ (6) avec $y_i = 1$

Remarquez que l'inéquation (6) est la même que l'inéquation (4). La même inéquation est donc obtenue pour les exemples positifs et les exemples négatifs. Ainsi, on peut résumer les deux inéquations (4) et (5) comme suit :

$y_i(\vec{w}.\vec{x}_i + b) - 1 \geq 0$

Pour les points les plus proches de l'hyperplan recherché, on impose la contrainte suivante $y_i(\vec{w}.\vec{x}_i + b) - 1 = 0$ (7).

Les points qui réalisent l'équation (7) sont ceux qui sont positionnés sur la frontière de la marge séparant la classe positive et la classe négative. Maximiser la marge entre ces deux classes revient à maximiser la quantité suivante :

$(\overrightarrow{x_+} - \overrightarrow{x_-}).\frac{\vec{w}}{||\vec{w}||} = \frac{(\overrightarrow{x_+}\ \vec{w} - \overrightarrow{x_-}\ \vec{w})}{||\vec{w}||}$ (8) (voir la figure 11-4)

© Editions ENI - All rights reserved

En utilisant l'équation (7), on obtient :

$\overrightarrow{x_+}.\vec{w} = 1 - b$, car $y_i = 1$ et $x+$ est un exemple positif.

$\overrightarrow{x_-}.\vec{w} = -b - 1$, car $y_i = -1$ et $x-$ est un exemple négatif.

Donc (8) devient $\frac{1-b+1+b}{||\vec{w}||} = \frac{2}{||\vec{w}||}$.

Le but est de maximiser la quantité $\frac{2}{||\vec{w}||}$ qui correspond à la marge séparant les exemples positifs des exemples négatifs. Pour des besoins de simplification mathématique, la quantité $\frac{2}{||\vec{w}||}$ sera remplacée par $\frac{1}{2}||\vec{w}||^2$.

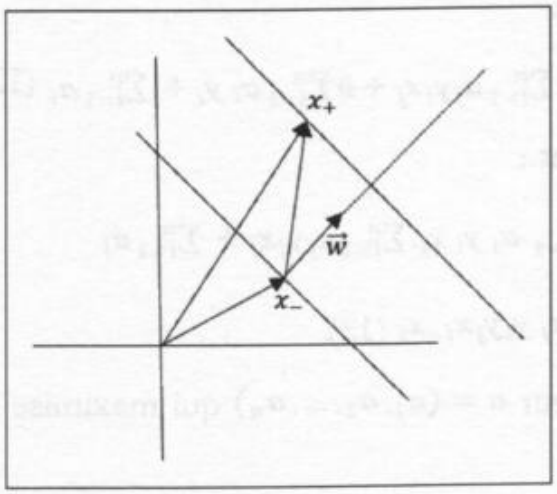


Figure 11-4 : la distance qui sépare les points positifs des points négatifs

En résumé, le problème à résoudre pour trouver le séparateur à vaste marge est le suivant $\begin{cases} Min\ \frac{1}{2}||\vec{w}||^2 \\ y_i(\vec{w}.\vec{x}_i + b) - 1 = 0 \end{cases}$

Pour résoudre ce problème, la méthode du dual, en introduisant des multiplicateurs de Lagrange, peut être utilisée.

Après l'application de cette méthode, le problème est reformulé comme suit :

$$L(w,\alpha) = \frac{1}{2}\left|\left|\vec{w}\right|\right|^2 - \sum_{i=1}^{n} \alpha_i [y_i(\vec{w}.\vec{x_i} + b) - 1] \quad (9)$$

Pour rechercher le maximum de la fonction L, il faudrait rechercher les points qui annulent les dérivées par rapport à w et b.

$$\frac{dL}{d\vec{w}} = \vec{w} - \sum_{i=1}^{n} \alpha_i y_i x_i = 0 \Rightarrow \vec{w} = \sum_{i=1}^{n} \alpha_i y_i x_i \quad (10)$$

$$\frac{dL}{db} = -\sum_{i=1}^{n} \alpha_i y_i = 0 \Rightarrow \sum_{i=1}^{n} \alpha_i y_i = 0 \quad (11)$$

L'équation (10) montre que $\vec{w}$ recherché n'est rien d'autre qu'une combinaison linéaire des exemples de la base d'apprentissage. En remplaçant $\vec{w}$ obtenu en (10) dans l'équation (9), on obtient :

$$\frac{1}{2}\sum_{i=1}^{n} \alpha_i y_i x_i . \sum_{j=1}^{n} \alpha_j y_j x_j - \sum_{i=1}^{n} \alpha_i y_i x_i . \sum_{i=1}^{n} \alpha_j y_j x_j + b \sum_{i=1}^{n} \alpha_i y_i + \sum_{i=1}^{n} \alpha_i \quad (12)$$

En utilisant (11), l'équation (12) devient :

$$L(w,\alpha) = \frac{1}{2}\sum_{i=1}^{n} \alpha_i y_i x_i . \sum_{j=1}^{n} \alpha_j y_j x_j - \sum_{i=1}^{n} \alpha_i y_i x_i . \sum_{i=1}^{n} \alpha_j y_j x_j + \sum_{i=1}^{n} \alpha_i$$

$$\Rightarrow \quad L(w,\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i . x_j \quad (13)$$

L'objectif est donc de trouver un vecteur $\alpha = (\alpha_1, \alpha_2, \ldots, \alpha_n)$ qui maximise la fonction $L(w,\alpha)$ définie en (13).

Le problème d'optimisation à résoudre pour trouver l'hyperplan à vaste marge est le suivant :

$$\begin{cases} Max\ L(w,\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i . x_j \\ \text{Avec} \quad \sum_{i=1}^{n} \alpha_i y_i = 0 \quad \text{et} \quad \alpha_i \geq 0, \forall i \in \{12, \ldots, n\} \end{cases}$$

Ce problème montre que la recherche de l'hyperplan à vaste marge dépend des produits scalaires entre les vecteurs (les données) de la base d'apprentissage.

© Editions ENI - All rights reserved

■Remarque

Ne soyez pas inquiet si vous rencontrez des difficultés pour comprendre la formulation de ce dernier problème d'optimisation, car cela ne vous empêchera pas d'utiliser les algorithmes SVM pour réaliser des modèles de prédiction ! En effet, pour comprendre une telle formulation, il est nécessaire d'avoir des connaissances en optimisation, qui est une branche à part entière en mathématiques.

4. Données non linéairement séparables

Jusqu'à présent, dans ce chapitre, l'algorithme SVM tel qu'il a été présenté traite uniquement des données linéairement séparables. Ainsi, la figure 11-2 a montré que pour trouver la classe d'appartenance d'un nouveau point, le modèle SVM procède à la projection de ce point sur l'axe portant le vecteur normal de l'hyperplan qui sépare linéairement les données de la base d'entraînement.

Dans la plupart des problèmes posés dans la vraie vie, les données sont rarement linéairement séparables. Pour contourner ce problème, il existe une solution consistant à appliquer une transformation sur les données.

L'idée de cette transformation est de passer par un espace intermédiaire dans lequel les données sont linéairement séparables et dans lequel le SVM peut être appliqué exactement comme l'ont montré les sections précédentes, puis restituer les résultats dans l'espace d'origine.

La figure suivante donne un exemple très simple de données non linéairement séparables. En effet, aucune droite ne peut séparer correctement les cercles d'un côté et les triangles de l'autre côté.

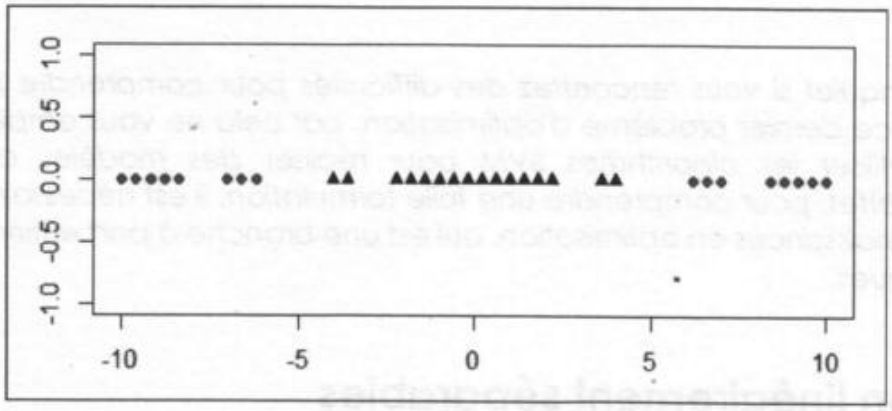


Figure 11-5 : exemple de données non linéairement séparables

Les données de cet exemple sont définies dans un espace à une dimension. Chaque point est associé à son abscisse x qui correspond à sa coordonnée sur un unique axe. La figure 11-6 donne le résultat de la transformation de ces points avec une fonction $\Phi(x)= (x, y = x^2)$. Les nouveaux points sont cette fois linéairement séparables !

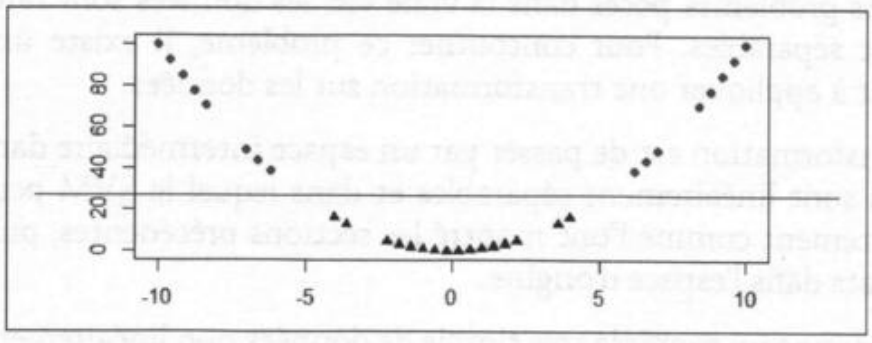


Figure 11-6 : transformation des points (x) par l'application $\Phi(x)= (x, y = x^2)$

Le but de cet exemple simple, mais pédagogique, est de montrer la possibilité d'utiliser une fonction Φ pour transformer des données non linéairement séparables et les rendre linéairement séparables.

Dans la réalité, les transformations Φ nécessaires pour rendre les données linéairement séparables sont plus complexes que la transformation définie dans cet exemple. Généralement, la fonction Φ transforme les données définies dans un espace à m_1 dimensions vers un autre espace à m_2 dimensions avec $m_2 > m_1$.

© Editions ENI - All rights reserved

Plus la fonction de transformation Φ est complexe, plus la résolution du problème d'optimisation pour trouver l'hyperplan à vaste marge devient compliquée. En effet, le problème à résoudre avec la transformation Φ devient le suivant :

$$P = \begin{cases} Max\ L(w,\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j\, y_i y_j\, \Phi(x_i).\Phi(x_j) \\ \\ \text{Avec } \sum_{i=1}^{n} \alpha_i y_i = 0 \text{ et } \alpha_i \geq 0, \forall\, i \in \{1,2,\dots,n\} \end{cases}$$

Également, avec une fonction de transformation Φ complexe, prédire la classe d'appartenance des données futures peut nécessiter des temps de calcul assez importants. En effet, une fois que w est fixé, prédire la classe d'appartenance d'une nouvelle donnée X revient à vérifier le signe de $Sign[\, w.X + b\,]$, voir l'équation (1)

En remplaçant w par sa valeur $\sum_{i=1}^{n} \alpha_i y_i x_i$, voir l'équation (10), on obtient :

$$Sign[\, \sum_{i=1}^{n} \alpha_i y_i x_i\, .X + b\,] = Sign[\, \sum_{i=1}^{n} \alpha_i y_i (x_i\, .X) + b\,]$$

Avec la transformation Φ, la dernière formule devient :

$$Sign[\, \sum_{i=1}^{n} \alpha_i y_i (\Phi(x_i)\, .\Phi(X)) + b\,]$$

Le temps de calcul de cette dernière formule dépend directement de la complexité de la fonction Φ.

4.1 Le Kernel Trick

L'idée d'appliquer une transformation sur les données pour sortir d'un espace de données non linéairement séparable et le rendre linéairement séparable est intéressante en soi. Cependant, comme évoqué à la section précédente, la complexité de cette transformation Φ peut causer un problème lié au coût des calculs associés à la résolution du problème d'optimisation P, et également au coût des calculs associés aux prédictions sur les données de production.

Pour contourner ce problème, il existe une solution permettant d'appliquer la transformation Φ sans connaître sa définition !

Par exemple, soit $X = (x, y)$ et $\Phi(x,y) = (x^2, \sqrt{2}\,x\,y, y^2)$, alors :

$$\begin{aligned}\Phi(x,y).\Phi(x',y') &= (x^2, \sqrt{2}\,x\,y, y^2).(x'^2, \sqrt{2}\,x'y', y'^2)\\ &= x^2x'^2 + \sqrt{2}\,x\,y\,\sqrt{2}\,x'y' + y^2y'^2\\ &= (xx')^2 + 2\,x\,x'y\,y' + (yy')^2\\ &= [(x,y).(x',y')]^2\end{aligned}$$

Pour calculer le produit scalaire $\Phi(x,y).\Phi(x',y')$, il n'est donc pas nécessaire de savoir que $\Phi(x,y) = (x^2, \sqrt{2}\,x\,y, y^2)$ et $\Phi(x',y') = (x'^2, \sqrt{2}\,x'\,y', y'^2)$. Il suffit de calculer $[(x,y).(x',y')]^2$ qui n'est finalement rien d'autre que le produit scalaire des deux vecteurs élevés au carré. C'est ce qu'on appelle l'astuce du noyau, communément appelée en anglais **Kernel Trick**.

De façon plus générale, au lieu de calculer toutes les transformations $\Phi(X)$ pour réaliser ensuite les produits scalaires $\Phi(X).\Phi(Y)$, il suffit de définir une fonction noyau $K(X,Y)$ simple à calculer, mais qui correspond en réalité à un produit scalaire dans un espace où les données sont linéairement séparables.

Avec cette astuce du noyau, le problème du SVM à résoudre devient :

$$P = \begin{cases} Max\ L(w,\alpha) = \sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j\,y_iy_j\,K(x_i,x_j) \\ \text{Avec}\ \ \sum_{i=1}^{n}\alpha_i y_i = 0 \ \ \text{et}\ \ \alpha_i \geq 0, \forall\, i \in \{1,2,\dots,n\} \end{cases}$$

Pour trouver la classe d'un nouveau point X, la solution est sous la forme :

$Sign[\sum_{i=1}^{n}\alpha_i y_i K(x_i.X) + b]$, voir l'équation (1)

© Editions ENI - All rights reserved

4.2 La condition de Mercer

L'utilisation d'une fonction noyau K permet donc de passer d'un espace où les données n'étaient pas linéairement séparables à un espace généralement beaucoup plus grand, où les données peuvent devenir linéairement séparables.

La section précédente a montré qu'avec le Kernel Trick la connaissance de la forme des données dans l'espace d'arrivée de la fonction K n'est pas nécessaire. En revanche, il faudrait s'assurer que cette même fonction K représente bien un produit scalaire dans cet espace d'arrivée.

Pour qu'une fonction $K: \mathrm{X} * \mathrm{X} \rightarrow \mathbb{R}$: soit une fonction noyau, il faut qu'elle vérifie la condition de Mercer dont l'énoncé est comme suit :

Une fonction symétrique $K(X,Y)$ est un noyau si :

$[K(X_i, X_j)]_{i,j}$ est une matrice semi-définie positive $\forall\, i, j$.

Si cette condition est vérifiée, alors $\exists\, \Phi: \mathrm{X} \rightarrow \mathrm{F}$ tels que $K(X,Y) = \Phi(\mathrm{X}).\Phi(\mathrm{Y})$

Cette condition, en pratique difficile à vérifier, garantit que $K(X,Y)$ représente bien un produit scalaire dans l'espace F.

Remarque

La condition de Mercer a été trouvée ou inventée par James Mercer en 1909.

4.3 Exemple de fonction noyau

Plusieurs fonctions noyaux existent. La liste non exhaustive ci-dessous recense les plus utilisées :

1. La fonction noyau polynomiale : $K(X,Y) = (X.Y + C)^n$.
2. La fonction noyau gaussienne : $K(X,Y) = \exp\left(-\frac{||X-Y||^2}{2\sigma^2}\right)$.
3. La fonction noyau sigmoïde : $K(X,Y) = \tanh\left((a(X.Y) - b)\right)$. La condition de Mercer n'est vérifiée que pour certaines valeurs de a et certaines valeurs de b.

5. Détecter les fraudes de cartes de crédit

Cet exemple concerne la réalisation d'un modèle pour détecter les fraudes de cartes de crédit. Il s'agit d'entraîner un modèle sur un ensemble de données contenant des transactions saines et des transactions frauduleuses.

5.1 Les données des transactions de cartes de crédit

Nous allons utiliser un jeu de données issu du site kaggle.com. Il s'agit d'un dataset du monde réel contenant 284807 transactions par carte de crédit. Parmi ces transactions, 492 représentent des fraudes. Vous trouverez ce jeu de données dans le fichier *creditcard.csv* sur le site https://www.kaggle.com/mlg-ulb/creditcardfraud.

Ce jeu de données est défini par vingt-huit variables anonymes $V_1, V_2, \ldots, V_{28}$ qui sont les résultats d'une ACP réalisée sur les données d'origine, plus trois autres variables `Time`, `Amount` et `Class` définies comme suit :

- `Time` : représente le temps en secondes passé depuis la première transaction du dataset.
- `Amount` : indique la somme d'argent concernée par la transaction.
- `Class` : est la variable binaire à expliquer. Elle indique si la transaction est frauduleuse ou pas. Elle vaut `0` si la transaction est normale, sinon elle vaut `1` si la transaction est frauduleuse.

Remarque

ACP est l'acronyme de l'Analyse en Composantes Principales, qui est une méthode permettant de résumer un ensemble de données. Le chapitre Les réseaux de neurones est entièrement consacré à cette méthode.

Remarque

Pour des raisons de confidentialité, les données à l'origine de l'ACP ne sont pas divulguées sur le site kaggle.com.

Il faut noter d'ores et déjà que la variable `Class` est très mal équilibrée, car le taux des transactions frauduleuses est très faible, il est égal à 0.172 %.

© Editions ENI - All rights reserved

Par conséquent, l'utilisation de l'indicateur AUC pour évaluer un quelconque modèle sur ce jeu de données ne sera pas appropriée. Dans ce cas, c'est l'utilisation de la courbe Précision/Recall qui sera recommandée.

Ce jeu de données a été publié dans : *Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson and Gianluca Bontempi. Calibrating Probability with Undersampling for Unbalanced Classification.In Symposium on Computational Intelligence and Data Mining (CIDM), IEEE, 2015.*

5.2 Application de l'algorithme SVM pour la détection des transactions bancaires frauduleuses

5.2.1 Application de l'algorithme SVM sur les données creditcard.csv

- Comme dans les précédents chapitres, lorsque vous avez appliqué des algorithmes tels que les algorithmes de régression et ceux de type Random Forest, pour appliquer un algorithme SVM, commencez par importer les bibliothèques nécessaires comme le montre le code suivant ; vous pouvez utiliser le code contenu dans le notebook *svm-01.ipynb* du dossier Code de ce chapitre :

```
import pandas as pd
from sklearn.metrics import precision_recall_fscore_support as
score
from sklearn.model_selection import train_test_split
from sklearn  import svm
import numpy as np
```

La nouveauté de ce code est l'import de la bibliothèque `svm` qui, comme son nom l'indique, contient les fonctions nécessaires pour l'utilisation de l'algorithme SVM.

- Chargez les données contenues dans le fichier creditcard.csv pour les stocker dans le DataFrame `data_origin` avec l'instruction suivante :

```
data_origin = pd.read_csv('../Data/creditcard.csv', sep=',')
```

Tout au long de ce chapitre, nous allons appliquer l'algorithme SVM plusieurs fois en apportant à chaque fois des modifications aux données et/ou aux paramètres du SVM. L'objectif est de montrer l'impact de ces modifications sur la qualité des modèles obtenus.

Dans un premier temps, nous allons appliquer l'algorithme SVM sur les données dans leur état d'origine.

Pour cela, avec l'instruction suivante, créez un nouveau DataFrame `data` dans lequel vous stockez toutes les variables explicatives :

```
data= data_origin[['Time', 'Amount', 'V1', 'V2', 'V3', 'V4',
'V5', 'V6', 'V7', 'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14',
'V15', 'V16', 'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23',
'V24', 'V25', 'V26', 'V27', 'V28']].copy()
```

Remarque

L'objectif de créer un nouveau DataFrame est de conserver dans la variable `data_origin` les données dans leur état d'origine afin qu'elles soient disponibles lors des prochaines applications de l'algorithme SVM, car vous pouvez réaliser tous les exemples de code de ce chapitre dans un seul et même notebook.

- De la même manière, stockez les données de la variable à expliquer dans un DataFrame y à l'aide de l'instruction suivante :

```
y =data_origin['Class']
```

- Puis créez le jeu de données d'entraînement et le jeu de données de test avec l'instruction suivante :

```
x_train, x_test, y_train, y_test = train_test_split(data, y,
test_size=0.2)
```

Ici, nous avons spécifié que 20 % des données seront stockées dans le jeu de données de test composé par `x_test` et `y_test` et donc 80 % des données seront stockées dans le jeu de données d'entraînement `x_train` et `y_train`.

Pour vous convaincre que ces données sont bien séparées avec ces proportions, les quatre instructions suivantes affichent la taille des données d'entraînement et de test :

```
print('La taille des données d\'entraînement X =
{}'.format(np.shape(X_train)))

print('La taille des données d\'entraînement Y =
{}'.format(np.shape(Y_train)))

print('La taille des données de test X =
```

© Editions ENI - All rights reserved

```
{}'.format(np.shape(X_test)))

print('La taille des données de test Y =
{}'.format(np.shape(Y_test)))
```

Le résultat de l'exécution de la cellule Jupyter précédente est le suivant :

```
La taille des données d'entraînement X = (227845, 30)
La taille des données d'entraînement Y = (227845,)
La taille des données de test X = (56962, 30)
La taille des données de test Y = (56962,)
```

Cela nous permet de vérifier que le DataFrame `x_train` contient bien 30 variables et 227845 données de transactions bancaires et `y_train` leurs libellés associés et que le DataFrame `x_test` contient également 30 variables correspondant à 56962 transactions bancaires et `y_test` leurs libellés associés.

Un point très important à vérifier également lors de la création des jeux de données d'entraînement et de test est la répartition des observations positives entre ces deux jeux de données. Ce contrôle peut être réalisé en affichant le nombre de cas positifs présents dans `y_train` et le nombre de cas positifs conservés dans le jeu de données de test `y_test`. En d'autres termes, il est important de savoir comment les cas positifs recherchés sont répartis entre les données d'entraînement et les données de test.

Pour notre exemple, nous pouvons afficher le nombre de transactions frauduleuses contenues dans `y_train` et celles contenues dans `y_test`.

▶Pour cela, utilisez les deux instructions suivantes :

```
print('Transactions frauduleuses dans y_train = ',sum(y_train))
print('Transactions frauduleuses dans y_test = ',sum(y_test))
```

Voici le résultat de l'exécution de ce code :

```
Transactions frauduleuses dans y_train =  395
Transactions frauduleuses dans y_test =  97
```

On constate que le rapport d'environ 20 % a bien été respecté par la procédure de séparation des données réalisée avec la fonction `train_test_split`.

Il faut noter que ce résultat obtenu avec la fonction `train_test_split` peut être légèrement différent de celui que vous obtenez.

Maintenant que le jeu de données est prêt, nous allons appliquer l'algorithme SVM. L'instruction suivante permet de créer une instance de l'algorithme SVM :

```
alg_svm= svm.SVC(kernel = 'linear')
```

À ce stade, nous disposons d'un objet `alg_svm`, qui représente notre algorithme SVM et qui ne demande qu'à ce qu'on lui fournisse les données d'entraînement.

Remarquez la valeur `linear` affectée à l'argument `kernel` lors de la construction de l'objet `alg_svm` associé à notre algorithme SVM. La valeur `linear` de l'argument `kernel` permet d'indiquer que c'est la version linéaire de l'algorithme SVM qui sera utilisée. En d'autres termes, l'algorithme SVM sera appliqué sur des données qui sont supposées être linéairement séparables.

Dans la suite de cette section, nous allons appliquer des versions de l'algorithme SVM avec des valeurs autres que `linear` pour le paramètre `kernel`.

L'instruction suivante permet de lancer la procédure d'apprentissage sur le jeu de données d'entraînement défini par les DataFrames `x_train` et `y_train` :

```
alg_svm.fit(x_train, y_train)
```

L'exécution de l'instruction précédente peut prendre plusieurs minutes, car le jeu de données d'apprentissage est assez important puisqu'il est composé de 227845 individus, chaque individu étant défini par 30 variables explicatives.

À la fin de cette phase d'apprentissage, le résultat suivant s'affiche :

```
SVC(kernel='linear')
```

L'exécution de la fonction `fit` se termine par l'affichage de la valeur utilisée avec le paramètre `kernel`.

À ce stade, le modèle a bien défini un hyperplan qui sépare les deux classes associées aux données d'entraînement. Comme nous l'avons vu au début de ce chapitre, cet hyperplan est principalement défini par les vecteurs supports ! Vous pouvez accéder à l'ensemble de tous les vecteurs supports en utilisant l'attribut `support_vectors_` de l'objet `alg_svm`.

© Editions ENI - All rights reserved

L'instruction suivante affiche le nombre des vecteurs supports trouvés par notre algorithme :

```
print(f"Le nombre de vecteurs supports est de :
{len(alg_svm.support_vectors_)}")
```

Dans notre exemple, la dernière instruction affiche le résultat suivant :

```
Le nombre de vecteurs supports est de : 533
```

Bien sûr, ce nombre de vecteurs supports dépend directement du jeu de données d'entraînement et des paramètres de l'algorithme SVM. Si vous exécutez de nouveau le code précédent à partir de la cellule Jupyter contenant l'appel de la fonction `train_test_split`, vous allez fort probablement aboutir à un jeu de données différent de celui associé à l'exécution précédente de ce même code. Cette différence dans le jeu de données d'entraînement, si minime soit-elle, peut engendrer un nombre de vecteurs supports différent après l'exécution de la fonction `fit`.

Nous pouvons même afficher le nombre des vecteurs supports par classe, par exemple avec l'instruction suivante :

```
print(f"Le nombre de vecteurs supports par classe :
{alg_svm.n_support_}")
```

Cette instruction affiche le résultat suivant :

```
Le nombre de vecteurs supports par classe : [266 267]
```

Pour notre exemple, le premier élément de la liste affichée correspond à la classe `0` et le deuxième élément correspond à la classe `1` ; d'ailleurs, 266 + 267 = 533. Pour en avoir le cœur net, vous pouvez utiliser l'instruction suivante :

```
print(f"Les classes ou les catégories manipulées sont :
{alg_svm.classes_}")
```

Ce qui produit ce résultat :

```
Les classes ou les catégories manipulées sont : [0 1]
```

L'étape de l'apprentissage sur les données d'entraînement est terminée. Nous allons donc pouvoir utiliser le modèle résultant afin d'effectuer des prédictions sur le jeu de données de test défini par `x_test` et `y_test`.

L'instruction suivante permet de réaliser des prédictions sur ce jeu de données de test :

```
predictions = alg_svm.predict(x_test)
```

La variable `predictions` est tout simplement la liste des prédictions du modèle sur les données de test `x_test`.

L'instruction suivante permet de calculer le nombre de cas positifs sur le jeu de données de test estimé par notre modèle.

```
print('Le nombre de cas positifs est : ', sum(predictions))
```

Ce qui produit le résultat suivant :

```
Le nombre de cas positifs est : 40
```

Bien sûr, il est possible que ce résultat soit légèrement différent de celui que vous obtenez avec votre code.

Pour évaluer notre modèle en nous basant sur les prédictions réalisées sur le jeu de données de test, nous utilisons les deux métriques de performance `precision` et `recall`. Pour calculer la `precision` et le `recall` nous allons utiliser la fonction `score` comme suit :

```
precision, recall, _, _ = score(y_test, predictions, pos_label=1,
average='binary')
```

La fonction `score` confronte tout simplement les prédictions réalisées par notre modèle sur les données de test et qui sont stockées dans le tableau `predictions` avec la réalité renseignée dans `y_test`.

On remarque le paramètre `pos_label` positionné à la valeur `1` afin d'indiquer la valeur qui sera utilisée pour le calcul des valeurs du `recall` et de `precision`, c'est-à-dire que la classe des cas positifs est désignée par la valeur `1`.

Nous allons également calculer le taux des prédictions correctes global `accuracy` à l'aide de l'instruction suivante :

```
accuracy = round((predictions==y_test).sum() /
len(predictions),3)
```

© Editions ENI - All rights reserved

À l'exécution de cette dernière instruction, la variable `accuracy` contient le taux de succès global obtenu par notre modèle.

L'instruction suivante permet enfin d'afficher nos trois métriques de performance en indiquant que les résultats doivent être affichés avec un arrondi de la partie décimale à trois positions après la virgule :

```
print('Precision: {} / Recall: {}./ Accuracy:
{}'.format(round(precision, 3), round(recall, 3), accuracy))
```

Le résultat de la dernière instruction est le suivant :

```
Precision: 0.7 / Recall: 0.289 / Accuracy: 0.999
```

On voit que le `recall` est très faible. Cela signifie que seulement 28.9 % des transactions frauduleuses contenues dans le jeu de données de test ont été détectées par notre modèle.

La valeur de `precision` est égale à 70 %. Cela correspond à la probabilité qu'une transaction soit réellement frauduleuse lorsqu'elle est détectée comme telle par notre modèle.

On remarque que le taux de succès global de réussite de notre modèle est très élevé, il est égal à 99.9 %. Malgré cela, les niveaux respectifs des métriques `recall` et `precision` sont relativement faibles. Cela est lié au fait que la variable à expliquer est déséquilibrée, car le nombre de transactions normales est beaucoup plus important que le nombre de transactions frauduleuses.

- Saisissez dans une cellule Jupyter le nom de la variable `alg_svm` suivi d'un point, puis appuyez sur la touche [Tab] de votre clavier pour afficher et explorer les différents attributs de cet objet ainsi que les différents paramètres utilisés par l'algorithme lors de la phase d'apprentissage.

Remarque

Remarquez au passage le nombre assez important de ces paramètres. Notez également dès maintenant que tous ces paramètres ne sont pas compatibles entre eux, dans le sens où certains ne sont utilisables que sous certaines conditions. Par exemple, le paramètre `Gamma` ne sera pas utilisé par l'algorithme si la valeur `linear` est affectée au paramètre `kernel`. Nous allons aborder un peu plus en détail ces paramètres dans la suite de ce chapitre.

5.2.2 Application du SVM sur un sous-ensemble de creditcard.csv

Les données sur lesquelles nous avons appliqué le SVM dans l'exemple précédent présentent une anomalie que nous pouvons remarquer par une simple visualisation des données du fichier *creditcard.csv*. En effet, on peut facilement voir que les deux variables `Time` et `Amount` sont définies dans des intervalles beaucoup plus larges que les vingt-huit autres variables.

Dans ce nouvel exemple, nous allons appliquer l'algorithme SVM exactement de la même manière que dans l'exemple précédent, sauf que cette fois-ci nous n'allons pas inclure ces deux variables `Time` et `Amount`. Le code suivant, défini dans le notebook *svm-02.ipynb* du dossier *Code* de ce chapitre reprend globalement les mêmes instructions que le code précédent :

```
import pandas as pd
from sklearn.metrics import precision_recall_fscore_support as
score
from sklearn.model_selection import train_test_split
from sklearn  import svm
import numpy as np

data_origin = pd.read_csv('../Data/creditcard.csv', sep=',')

data= data_origin[['V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7',
'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14', 'V15', 'V16',
'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23', 'V24', 'V25',
'V26', 'V27', 'V28']].copy()

y =data_origin['Class']

x_train, x_test, y_train, y_test = train_test_split(data, y,
test_size=0.2)

alg_svm= svm.SVC(kernel = 'linear')

alg_svm.fit(x_train, y_train)

predictions = alg_svm.predict(x_test)

precision, recall, _, _ = score(y_test, predictions, pos_label=1,
average='binary')

accuracy = round((predictions==y_test).sum()/len(predictions),3)
```

© Editions ENI - All rights reserved

```
print('Precision: {} / Recall: {} / Accuracy:
{}'.format(round(precision, 3), round(recall, 3), accuracy))

print('Le modèle prédit {} transactions
frauduleuses'.format(sum(predictions)))

print('Dans les données de test, il y a {} transactions
frauduleuses'.format(sum(Y_test)))
```

Les seules différences avec le code précédent résident dans la définition de la variable `data` qui cette fois-ci ne contient pas les deux variables `Time` et `Amount`.

L'exécution de ce nouveau programme donne les résultats suivants :

```
Precision: 0.827 / Recall: 0.827 / Accuracy: 0.999
Le modèle prédit 104 transactions frauduleuses
Dans les données de test, il y a 104 transactions frauduleuses
```

La première chose que nous pouvons remarquer est que sans l'utilisation des deux variables `Time` et `Amount`, les deux métriques de performance `recall` et `precision` sont nettement améliorées !

Remarque

Ces résultats peuvent être surprenants, mais attention, cela ne signifie pas que les deux variables que nous avons exclues de cette nouvelle expérimentation n'ont pas de pouvoir prédictif pour la détection des transactions frauduleuses. De plus, dans les deux exemples précédents, nous avons posé l'hypothèse que les transactions, notamment les transactions frauduleuses, sont complètement indépendantes les unes par rapport aux autres et qu'il n'y a aucun lien d'ordre chronologique entre elles. C'est du fait de cette hypothèse que nous n'avons pas considéré ce jeu de données comme étant une série temporelle dont l'ordre chronologique serait défini par la variable `Time`. Dans la réalité, il peut être intéressant d'exploiter un tel jeu de données en tant que série temporelle ! Nous ne l'avons pas fait dans nos exemples afin de nous concentrer sur l'application du SVM de la façon la plus simple et la plus pédagogique possible.

Afin de renforcer votre compréhension des deux métriques de performance `recall` et `precision`, un point intéressant à observer dans ces nouveaux résultats est que le nombre de transactions réellement frauduleuses dans le jeu de données de test est égal à 104 et le modèle a en effet prédit 104 transactions frauduleuses. Cependant, cela ne signifie pas que ce sont les 104 transactions réellement frauduleuses qui ont été détectées par le modèle ; et cela se confirme par un `recall` égal à 82.7 % et non pas égal à 100 % ! Bien sûr, dans cet exemple, le fait que le nombre de transactions réellement frauduleuses soit égal au nombre de transactions frauduleuses prédites par le modèle est un simple hasard !

5.2.3 Application du SVM sur des données normalisées

Dans les deux exemples précédents, nous avions appliqué l'algorithme SVM pour construire un modèle de détection des transactions bancaires frauduleuses sans nous poser de questions sur la nature des distributions associées aux variables qui définissent le jeu de données utilisé.

En effet, dans le premier exemple, nous avons utilisé les données dans leur état d'origine, et dans le deuxième exemple nous avons tout simplement exclu les deux variables `Time` et `Amount` sans pour autant analyser ces variables en détail.

Si nous prêtons un peu plus d'attention à la forme de nos données, nous remarquons les différences des échelles de définitions entre les deux variables `Time` et `Amount` et les vingt-huit autres variables, en allant de `V1` jusqu'à `V28`. Ces différences d'échelles entre les variables posent bien sûr un gros problème puisque l'algorithme SVM utilise les valeurs des variables pour détecter les vecteurs supports et ainsi tracer les frontières entre les cas positifs et négatifs, c'est-à-dire délimiter les frontières entre les transactions frauduleuses et les transactions normales. Ce décalage d'échelle de définition des variables peut facilement conduire à des erreurs de calcul importantes.

© Editions ENI - All rights reserved

Jetons un coup d'œil à la nature des distributions associées à nos variables.

- Commencez avec le code suivant, par la lecture du DataFrame `data` comme lors du premier exemple, et créez un nouveau notebook Jupyter ou utilisez le code du notebook *stats.ipynb* contenu dans le dossier Code de ce chapitre :

```
import pandas as pd
from matplotlib import pyplot
import math
%matplotlib inline

data_origin = pd.read_csv('../Data/creditcard.csv', sep=',')
data= data_origin[['Time','Amount','V1', 'V2', 'V3', 'V4', 'V5',
'V6', 'V7', 'V8', 'V9', 'V10',
       'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18',
'V19', 'V20',
       'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27',
'V28']].copy()
```

Nous allons utiliser la fonction `describe()` afin de visualiser dans une seule table les statistiques de base associées à toutes les variables. L'instruction suivante permet d'afficher la table de description de nos données :

```
data.describe()
```

Le résultat de l'instruction précédente est résumé dans la table suivante :

	Time	Amount	V1	V2	V3	V4	V5	V6	V7	V8
count	284807.000000	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05
mean	94813.859575	88.349619	3.919560e-15	5.688174e-16	-8.769071e-15	2.782312e-15	-1.552563e-15	2.010663e-15	-1.694249e-15	-1.927028e-16
std	47488.145955	250.120109	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.380247e+00	1.332271e+00	1.237094e+00	1.194353e+00
min	0.000000	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.137433e+02	-2.616051e+01	-4.355724e+01	-7.321672e+01
25%	54201.500000	5.600000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.915971e-01	-7.682956e-01	-5.540759e-01	-2.086297e-01
50%	84692.000000	22.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.433583e-02	-2.741871e-01	4.010308e-02	2.235804e-02
75%	139320.500000	77.165000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.119264e-01	3.985649e-01	5.704361e-01	3.273459e-01
max	172792.000000	25691.160000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.480167e+01	7.330163e+01	1.205895e+02	2.000721e+01

8 rows × 30 columns

Figure 11-7 : les statistiques de base des variables `Time`, `Amount` et des variables de V1 à V28

Une simple visualisation de cette table permet de constater que la variable `Time` est définie dans l'intervalle `[min, max]=[0, 172792]` et la variable `Amount` est définie dans l'intervalle `[min, max]=[0, 25691.16]`, alors que toutes les autres variables sont définies avec des valeurs très proches de `0`. En effet, les variables de `V1` à `V28` ont chacune pour moyenne une valeur très proche de la valeur `0`.

Par exemple, si nous visualisons dans un histogramme la distribution de la variable `V1` avec le code suivant, nous obtenons le diagramme de la figure ci-dessous :

```
bins = [-60,-50,-40,-30,-20,-10,-9,-8,-7,-6,-5,-4,-3,-2,-1,0,1,2]
pyplot.figure(figsize=(10,7))
pyplot.hist(data['V1'], bins, alpha=0.5,label='V1')
pyplot.legend(loc='upper right')
pyplot.show()
```

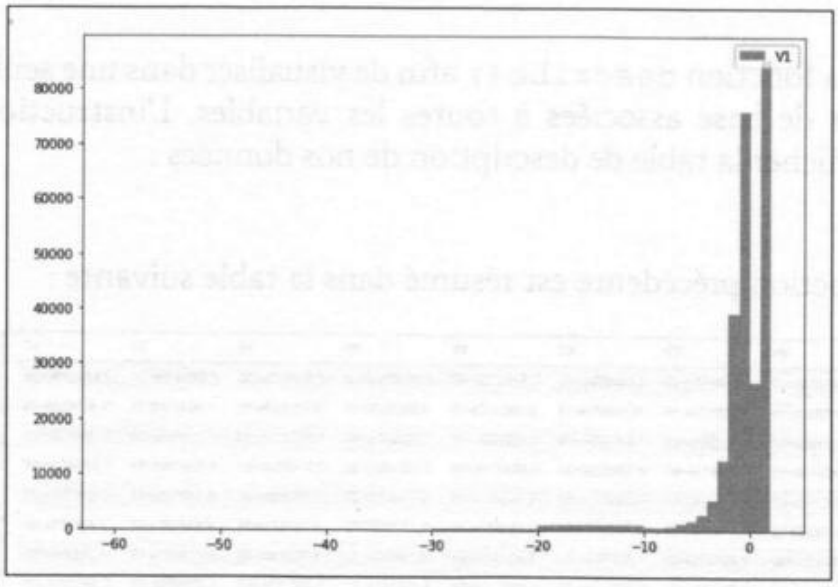


Figure 11-8 : l'histogramme associé à la variable `V1`

Et si nous visualisons le diagramme de la variable `Amount` avec le code suivant, nous obtenons le résultat sur la figure 11-9 ci-après :

```
bins = [0,5,10,15,20,25,30,50,75,100,125,150,175,200,300,400,500]
pyplot.figure(figsize=(10,7))
pyplot.hist(data['Amount'], bins, alpha=0.5,  label='Amount')
pyplot.legend(loc='upper right')
pyplot.show()
```

© Editions ENI - All rights reserved

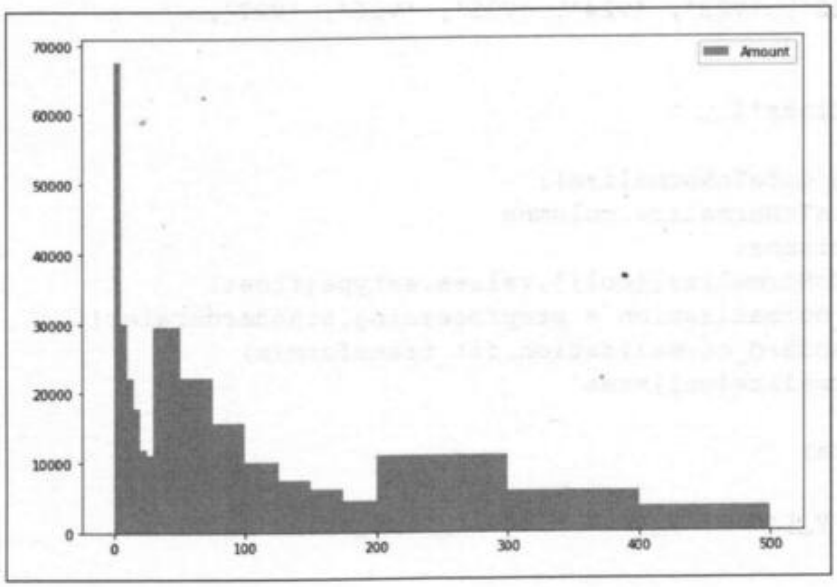


Figure 11-9 : l'histogramme associé à la variable `Amount`

Les deux diagrammes précédents montrent clairement et visuellement la différence des deux distributions associées aux deux variables `Amount` et `V1` et nous obtenons des différences similaires si nous comparons cette variable `Amount` avec chacune des variables de `V2` à `V28`.

Dans l'exemple suivant, nous allons procéder à la normalisation de toutes les variables du jeu de données du fichier *creditcard.csv*, sauf la variable `Time` qui ne sera pas utilisée. Puis nous allons appliquer l'algorithme SVM exactement de la même manière que dans l'exemple précédent.

- Pour cela, utilisez le code suivant, créez un nouveau notebook ou utilisez le notebook *svm-03.ipynb* contenu dans le dossier *Code* de ce chapitre :

```
import pandas as pd
from sklearn.metrics import precision_recall_fscore_support as score
from sklearn.model_selection import train_test_split
from sklearn  import svm
import numpy as np
from sklearn import preprocessing

data_origin = pd.read_csv('../Data/creditcard.csv', sep=',')
data= data_origin[['Amount','V1', 'V2', 'V3', 'V4', 'V5', 'V6',
'V7', 'V8', 'V9', 'V10',
       'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18',
'V19', 'V20',
```

```
        'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27',
'V28']].copy()

y =data_origin['Class']

def normalization(dataToNormalize):
    columns = dataToNormalize.columns
    for col in columns:
        x = dataToNormalize[[col]].values.astype(float)
        standard_normalization = preprocessing.StandardScaler()
        res = standard_normalization.fit_transform(x)
        dataToNormalize[col]=res

normalization(data)

x_train, x_test, y_train, y_test = train_test_split(data, y,
test_size=0.2)

alg_svm= svm.SVC(kernel = 'linear')

alg_svm.fit(x_train, y_train)

predictions = alg_svm.predict(x_test)

precision, recall, _, _ = score(y_test, predictions, pos_label=1,
average='binary')

accuracy = round((predictions==y_test).sum() / len(predictions),3)

print('Precision: {} / Recall: {} / Accuracy: {}'.format(round(preci-
sion, 3), round(recall, 3), accuracy))

print('Le modèle prédit {} transactions
frauduleuses'.format(sum(predictions)))

print('Dans les données de test, il y a {} transactions
frauduleuses'.format(sum(y_test)))
```

La seule différence entre ce nouveau code et le code de l'exemple précédent réside dans la définition et l'utilisation de la fonction `normalization`.

© Editions ENI - All rights reserved

Cette fonction normalise les colonnes d'un DataFrame qu'elle reçoit en argument comme suit :

- Récupération de toutes les variables du DataFrame reçu en paramètre.
- Pour chaque valeur de chaque variable, cette fonction soustrait la moyenne de la variable en question et divise le résultat par l'écart-type de cette même variable.

Mis à part la normalisation des variables, le reste de ce dernier code est similaire au code précédent. Une exécution de ce code a donné le résultat ci-après :

```
Precision: 0.867 / Recall: 0.802 / Accuracy: 0.999
Le modèle prédit 98 transactions frauduleuses
Dans les données de test, il y a 106 transactions frauduleuses
```

5.3 Les paramètres de l'algorithme SVM

L'algorithme SVM utilise plusieurs paramètres et la quête qui permet de trouver les valeurs optimales pour tous ces paramètres n'est pas une tâche facile. En réalité, les meilleures valeurs possibles pour les paramètres d'un algorithme SVM dépendent directement des données utilisées. De ce fait, il est nécessaire de réaliser plusieurs tests et de sélectionner les valeurs de ces paramètres qui permettent d'aboutir au meilleur modèle.

Dans la prochaine section, nous allons nous intéresser aux paramètres les plus couramment manipulés, dont la recherche du meilleur modèle. Il s'agit des paramètres `Kernel`, `C` et `Gamma`. Ces trois paramètres vont être expliqués, puis nous donnerons un exemple de code permettant d'utiliser *GridSearchCV* pour la recherche de valeurs intéressantes pour les paramètres `Kernel`, `C` et `Gamma`.

5.3.1 Le paramètre Kernel pour la variation de la fonction noyau

Le paramètre `Kernel` permet de désigner le type de la fonction noyau à utiliser. Les valeurs possibles pour ce paramètre sont les suivantes :

- `Linear` : la valeur que nous avons utilisée lors de tous les précédents exemples de ce chapitre. On l'utilise lorsque nous supposons que les données d'entraînement sont linéairement séparables et qu'aucune transformation de ces données n'est nécessaire. En d'autres termes, le Kernel Trick n'est pas nécessaire.
- `Poly` : la fonction noyau est une transformation polynomiale définie avec la formule suivante $(\gamma < x, y > +r)^d$

  Où γ, r et d sont trois paramètres spécifiés respectivement avec les arguments `Gamma`, `coef0` et `degree`.
- `Rbf` : cette fonction signifie *Radial Basis Function*, ou fonction à base radiale en français, et est définie comme suit $\exp(-\gamma||x-y||^2)$

  Avec γ un paramètre.

  Dans la littérature, on trouve souvent que cette fonction `Rbf` est définie par $\exp(-\frac{||X-Y||^2}{2\sigma^2})$

  Cependant, ces deux dernières formules sont en réalité les mêmes ; dans la documentation de Scikit-learn, le terme $\frac{1}{2\sigma^2}$ a simplement été remplacé par γ et c'est pour cette raison que dans la littérature on parle plutôt du paramètre γ et non du paramètre σ comme dans Scikit-learn.
- `Sigmoid` : cette fonction est définie comme suit $tanh(\gamma < x, y > +r)$

  Où γ et r sont deux paramètres spécifiés respectivement avec les arguments `Gamma` et `coef0`.
- Fonction définie par l'utilisateur : vous pouvez également définir votre propre fonction noyau et la transmettre en argument pour le constructeur d'instances de l'algorithme SVM, de la même manière que lorsque nous avons utilisé la valeur `linear` dans nos précédents exemples.

© Editions ENI - All rights reserved

Pour cela, il suffit de définir votre propre fonction permettant de réaliser le Kernel Trick. Cette fonction doit recevoir deux arguments correspondant à deux observations et retourne une valeur correspondant à la distance entre ces deux observations dans l'espace de plus grande dimension associé à votre fonction.

Remarque

Pour rappel, dans la mesure où votre fonction utilise la technique du Kernel Trick, vous n'avez nullement besoin de calculer les coordonnées de ces deux observations dans l'espace de plus grande dimension, cela est fait de manière implicite par la fonction que vous avez utilisée.

5.3.2 Le paramètre C

Nous avons vu que pour construire un modèle de classification binaire, l'algorithme SVM cherche un hyperplan le plus loin possible des observations des deux classes en question. La recherche d'un tel hyperplan passe par la résolution d'un problème d'optimisation où la bande séparatrice des deux classes est la plus vaste possible et où les données d'entraînement sont parfaitement classées. Cependant, dans certains cas, il serait plus intéressant de se satisfaire d'un hyperplan qui permet de classer correctement un grand nombre de données d'entraînement, mais pas toutes les données, et où la bande séparant les deux classes est de plus grande marge !

Considérons les points de la figure suivante représentés dans un plan à deux dimensions où les deux classes à séparer par un SVM sont la classe des carrés et la classe des cercles. Supposons que les données d'entraînement sont les carrés et les cercles vides et les données de production, ou les données de test, sont les carrés remplis.

Dans cet exemple, on voit que le modèle représenté avec la droite oblique sépare parfaitement les données d'entraînement, c'est-à-dire que les carrés et les cercles vides sont classés correctement à 100 %. En revanche, les carrés pleins sont malheureusement considérés comme étant des cercles puisqu'ils sont du côté gauche de la droite associée à ce modèle.

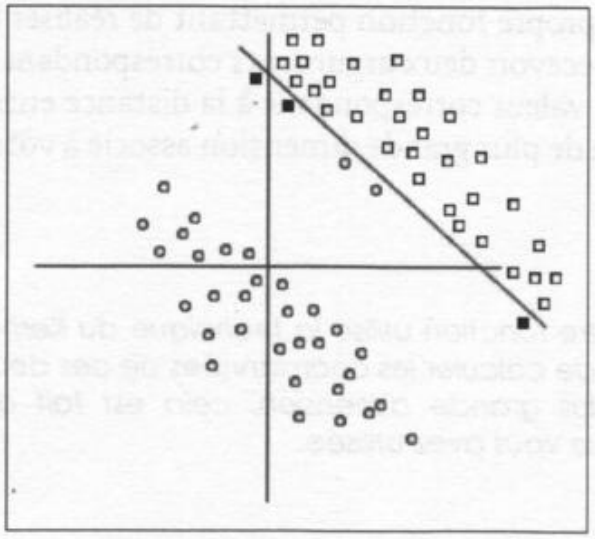


Figure 11-10 : exemple d'un modèle SVM non tolérant aux erreurs de classification

L'exemple précédent montre un cas où un modèle de type SVM peut avoir des performances maximales sur les données d'entraînement, mais des performances médiocres en production ! Généralement, nous faisons face à de telles situations lorsque nos modèles rencontrent le problème de surapprentissage.

Dans l'exemple précédent, on voit que pour obtenir un modèle avec un pouvoir de généralisation plus intéressant, nous devons construire un modèle où la marge séparant les carrés des cercles est plus vaste, au détriment d'une classification parfaite des données d'entraînement, comme dans l'exemple de la figure suivante.

© Editions ENI - All rights reserved

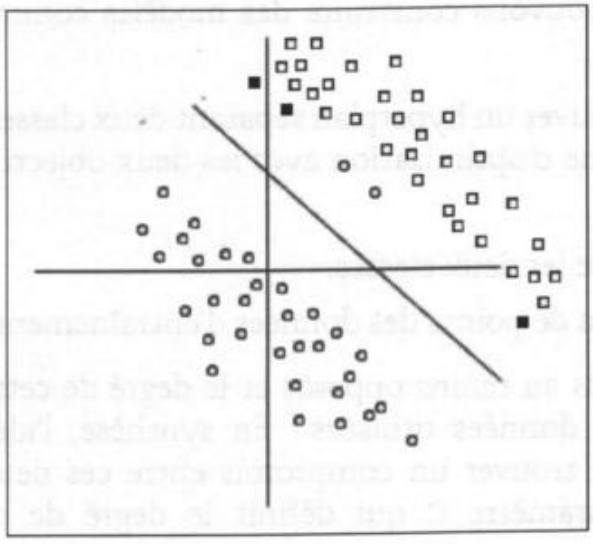


Figure 11-11 : exemple d'un modèle SVM tolérant aux erreurs de classification

En effet, cette dernière figure montre un modèle, représenté par la droite oblique, où les carrés et les cercles vides ne sont pas parfaitement séparés, puisqu'il y a deux cercles vides qui sont placés du mauvais côté de la droite, mais ce nouveau modèle a clairement un pouvoir prédictif beaucoup plus intéressant que le modèle précédent. Dans ce nouveau modèle, les carrés pleins sont cette fois-ci correctement classés.

Concrètement, l'algorithme SVM classique fonctionne de façon à trouver un hyperplan où toutes les données d'entraînement sont correctement classées au détriment du pouvoir de généralisation sur les données futures !

Par conséquent, lorsque nous utilisons l'algorithme SVM classique, nous supposons, ou nous espérons, que les données futures seront relativement loin des frontières tracées pas l'hyperplan correspondant au modèle. En effet, lorsque les données de test ou les données de production sont suffisamment loin de ces frontières, elles sont correctement classées malgré le fait que la marge séparatrice soit étroite. Par contre, si ces données sont très proches de ces mêmes frontières, alors le modèle risque de leur associer des classes erronées.

Pour pallier ce problème, en 1995, **Cortes** et **Vapnik** ont proposé une version du SVM, connue sous le nom de Soft Margin, où l'algorithme SVM accepte de mal classer certaines données d'entraînement en faveur de la construction d'un hyperplan à plus grande marge !

Avec cette nouvelle version, nous pouvons construire des modèles comme celui de l'exemple de la figure 11-11.

Avec la version Soft Margin, pour trouver un hyperplan séparant deux classes, l'algorithme SVM résout un problème d'optimisation avec les deux objectifs suivants :

- Accroître la marge séparatrice entre les deux classes.
- Classer correctement un maximum de points des données d'entraînement.

Bien sûr, ces deux objectifs sont plus au moins opposés et le degré de cette opposition dépend directement des données utilisées ! En synthèse, l'idée véhiculée par le Soft Margin est de trouver un compromis entre ces deux objectifs. C'est justement l'hyperparamètre `C` qui définit le degré de ce compromis.

Le paramètre `C` définit la pénalité associée à chaque observation d'entraînement mal classée. Par conséquent, nous pouvons déduire les deux points suivants :

1. Plus la valeur du paramètre `C` est élevée, moins large est la marge séparatrice, et le modèle tend à classer correctement un maximum de données, voire toutes les données d'entraînement. Dans ce cas, le risque est d'aboutir à un modèle avec des performances élevées sur les données d'entraînement, mais des performances médiocres sur les données de test et de production.
2. Plus la valeur du paramètre `C` est petite, plus grande est la marge séparatrice, puisque l'algorithme accepte plus facilement de mal classer certaines observations. Dans ce cas, le modèle n'est pas parfait sur les données d'entraînement, mais il a des capacités de généralisation plus intéressantes.

Il faut noter que la valeur du paramètre `C` n'est pas utilisée en tant que telle pour pénaliser une observation mal classée. La pénalité d'une observation mal classée se calcule en fonction de la valeur de ce paramètre `C` et de la distance de cette observation à l'hyperplan considéré. Les observations mal classées n'ont donc pas toutes la même pénalité.

© Editions ENI - All rights reserved

Pour revenir aux deux exemples précédents, une valeur trop élevée du paramètre C a une probabilité élevée d'aboutir au modèle de la figure 11-10 et une valeur faible a une probabilité importante d'aboutir au modèle de la figure 11-11.

5.3.3 Le paramètre Gamma

Le paramètre Gamma est utilisé uniquement avec des fonctions Kernel non linéaires et il permet d'indiquer le niveau d'influence de chaque observation du jeu de données d'entraînement sur la recherche de l'hyperplan associé au modèle. Lorsque la valeur de ce paramètre est élevée, pour que deux observations soient considérées comme faisant partie d'un même groupement de points, elles doivent être très proches entre elles. A contrario, lorsque la valeur du paramètre Gamma est suffisamment faible, les observations sont réparties dans des groupes de façon quasiment linéaire !

D'une certaine manière, ce paramètre Gamma permet de contrôler le niveau de courbure de l'hyperplan séparant les différentes classes. La figure 11-12 suivante montre une illustration du résultat d'un modèle qui serait obtenu avec une valeur du paramètre Gamma élevée.

Remarque

Pour rappel, ce type d'hyperplan avec des courbures n'existe pas et aucun SVM ne cherche réellement une fonction qui donne de telles courbures. Lorsque vous trouvez dans la littérature ce type d'hyperplan, il s'agit en réalité d'une représentation graphique des résultats retournés par un modèle SVM, car même lorsque les données ne sont pas linéairement séparables, le SVM utilise le Kernel Trick que nous avons abordé plus haut dans ce chapitre, afin de transformer ces données en données linéairement séparables, et cela sans pour autant calculer les nouvelles coordonnées des données dans l'espace de plus grande dimension.

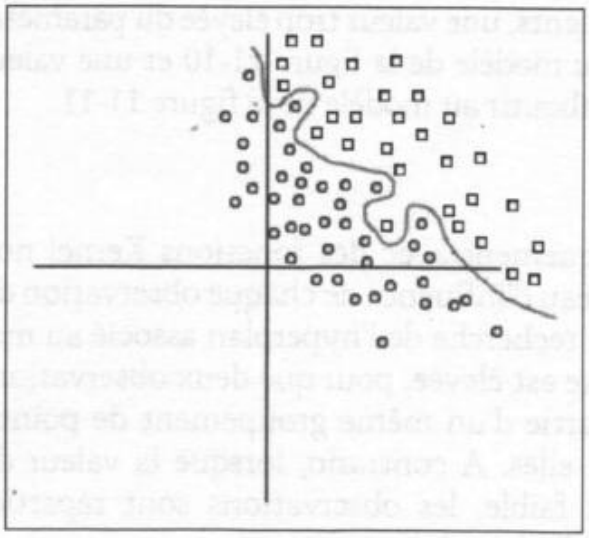


Figure 11-12 : exemple d'un modèle SVM avec une valeur élevée du paramètre Gamma

Les mêmes données que celles de la figure précédente peuvent aboutir à un hyperplan linéaire, comme dans la figure suivante, en utilisant une petite valeur du paramètre `Gamma`.

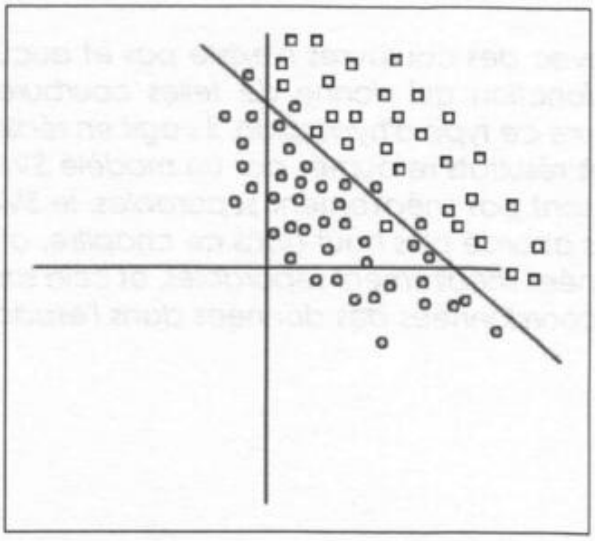


Figure 11-13 : exemple d'un modèle SVM avec une petite valeur du paramètre Gamma

Les deux figures précédentes montrent que plus la valeur de `Gamma` est élevée, plus le risque du surapprentissage est lui aussi élevé !

© Editions ENI - All rights reserved

5.3.4 Le paramètre C versus le paramètre Gamma

Nous avons vu que les deux paramètres `C` et `Gamma` influencent la forme de l'hyperplan associé à un modèle SVM. Lorsque la fonction Kernel utilisée est linéaire, nous avons simplement besoin d'optimiser la valeur du paramètre `C`, puisque le paramètre `Gamma` n'est pas applicable. En revanche, si la fonction Kernel utilisée est la fonction `Rbf`, alors nous avons réellement besoin d'optimiser les valeurs des deux paramètres `C` et `Gamma`. Cependant, si la valeur du paramètre `Gamma` est grande, alors le paramètre `C` devient négligeable.

Les valeurs communément utilisées avec ces deux paramètres `C` et `Gamma` sont les suivantes $\begin{matrix} 0.1 < C < 100 \\ 0.0001 < gamma < 10 \end{matrix}$.

Bien sûr, les valeurs optimales de ces deux paramètres dépendent directement des données utilisées et elles peuvent être différentes de celles mentionnées ici.

5.3.5 Tuning des hyperparamètres d'un SVM avec GridSearchCV

Afin d'illustrer le réglage d'hyperparamètres d'un algorithme SVM, dans ce nouvel exemple de construction d'un modèle de détection des transactions frauduleuses, nous allons appliquer cet algorithme SVM plusieurs fois avec à chaque fois des valeurs différentes de certains de ses paramètres en utilisant *GridSearchCV*.

- Pour la réalisation de cet exemple, créez un nouveau notebook et saisissez le code suivant ou utilisez le notebook *svm-04.ipynb* contenu dans le dossier *Code* de ce chapitre.
- Commencez par importer les modules suivants :

```
import pandas as pd
from sklearn.metrics import precision_recall_fscore_support as score
from sklearn.model_selection import train_test_split
from sklearn  import svm
import numpy as np
from sklearn import preprocessing
from sklearn.model_selection import GridSearchCV
```

Comme nous allons le voir dans la suite de ce nouveau programme, la classe `GridSearchCV` nous permet de construire de façon simple un pipeline où nous allons pouvoir spécifier un jeu de valeurs pour un ensemble de paramètres et appliquer un algorithme, qui sera un algorithme SVM dans notre cas, autant de fois que de combinaisons de ces valeurs de paramètres possibles. De plus, pour chaque combinaison de valeurs des paramètres spécifiés, l'algorithme choisi sera appliqué avec la technique de cross-validation. Le code suivant permet de lire et de normaliser les mêmes données que celles utilisées lors du dernier exemple. La dernière instruction de ce code permet tout simplement de construire les deux jeux de données d'entraînement (`x_train, y_train`) et de test (`x_test, y_test`).

```
data_origin = pd.read_csv('../Data/creditcard.csv', sep=',')
data= data_origin[['Amount','V1', 'V2', 'V3', 'V4', 'V5', 'V6',
'V7', 'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14', 'V15',
'V16', 'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23', 'V24',
'V25', 'V26', 'V27', 'V28']].copy()

y =data_origin['Class']

normalization(data)

x_train, x_test, y_train, y_test = train_test_split(data, y,
test_size=0.2)
```

Le code suivant permet de construire une instance d'un algorithme SVM avec le nom `alg_svm` et un dictionnaire `params` dans lequel nous avons indiqué trois informations, à savoir :

1. La clé `kernel` correspond à une liste contenant deux valeurs `rbf` et `linear`.

2. La clé `gamma` correspond à une liste de six valeurs.

3. La clé `C` correspond à une liste de six valeurs.

```
alg_svm= svm.SVC()
params = {'kernel': ['rbf'],
          'gamma': [0.0001,0.001,0.005,0.01,1,10],
          'C':[0.1,1,5,10,50,100]}
```

© Editions ENI - All rights reserved

Le dictionnaire `params` sera utilisé afin d'indiquer tous les paramètres avec lesquels nous souhaitons exécuter l'algorithme `alg_svm` ! Au total, il y aura 36 exécutions de cet algorithme en utilisant à chaque exécution des valeurs différentes des paramètres `Gamma` et `C` à partir de leurs listes respectives.

L'instruction suivante permet justement de construire une instance de l'objet `hyper_params_grid` qui va jouer le rôle d'un workflow. Pour cet objet de type `GridSearchCV`, nous avons indiqué les paramètres suivants :

- `alg_svm` pour indiquer que c'est cet algorithme, qui est un algorithme SVM, qui sera utilisé.
- `params` pour indiquer le jeu de valeurs des paramètres `kernel`, `gamma` et `C` à utiliser.
- `cv =2` afin de préciser le nombre de paquets qui seront utilisés avec la technique de cross-validation.
- `scoring='accuracy'` afin de choisir le critère de performance qui sera utilisé pour la comparaison des différents modèles qui seront construits.
- `n_jobs=-1` pour indiquer notre souhait d'utiliser tous les processeurs disponibles lors de la phase d'entraînement qui sera lancée plus tard avec la fonction `fit`.

```
hyper_params_grid = GridSearchCV(alg_svm, params, cv=2, scoring
='accuracy', n_jobs=-1)
```

Il ne nous reste plus qu'à lancer l'entraînement de l'algorithme `alg_svm` sur les données d'entraînement en utilisant la fonction `fit` comme dans l'instruction suivante :

```
hyper_params_models = hyper_params_grid.fit(x_train, y_train)
```

À la fin de l'exécution de la fonction `fit`, nous pouvons afficher les paramètres qui ont servi à la construction du meilleur modèle avec l'instruction suivante :

```
hyper_params_models.best_params_
```

En effet, l'affichage de l'attribut `best_params_` montre les résultats suivants :

```
{'C': 10, 'gamma': 0.005, 'kernel': 'rbf'}
```

Si nous n'avions pas spécifié des valeurs précises pour les deux paramètres `Gamma` et `C`, alors l'algorithme `alg_svm` aurait tout simplement utilisé les valeurs par défaut de ces deux paramètres et qui correspondent à la valeur `1` pour le paramètre `C` et à la valeur $\frac{1}{n_feature * X.var()}$ pour le paramètre `Gamma` où $n_feature$ est égal au nombre de variables et $X.var()$ la variance du jeu de données traité.

Ces derniers résultats montrent que les meilleures valeurs trouvées pour ces deux paramètres `Gamma` et `C` sont différentes de celles utilisées par défaut.

En effet, pour le paramètre `C`, c'est évident, car la meilleure valeur est égale à `10`, alors que la valeur par défaut est égale à `1`. Pour le paramètre `Gamma`, l'exécution du programme précédent sans fixer explicitement la valeur de `Gamma` s'est terminée en indiquant que la valeur du paramètre `Gamma` qui a été utilisée est d'environ `1.47`.

Remarque

Pour vérifier les valeurs par défaut de ces deux paramètres, il suffit d'exécuter un des exemples des sections précédentes, puis d'afficher les valeurs des attributs `alg_svm.C` et `alg_svm._gamma`.

L'instruction suivante nous permet justement de récupérer le meilleur modèle dans la variable `model`.

```
model = hyper_params_models.best_estimator_
```

Maintenant que nous avons ce meilleur modèle, nous n'avons qu'à l'utiliser pour réaliser les prédictions sur les données de test et afficher les métriques de performance `precision`, `recall` et `accuracy` comme nous l'avons fait lors des exemples précédents :

```
predictions = model.predict(x_test)

precision, recall, _, _ = score(y_test, predictions, pos_label=1,
average='binary')

accuracy = round((predictions==y_test).sum() /
len(predictions),3)

print('Precision: {} / Recall: {} / Accuracy:
```

© Editions ENI - All rights reserved

```
{}'.format(round(precision, 3), round(recall, 3), accuracy))

print('Le modèle prédit {} transactions
frauduleuses'.format(sum(predictions)))

print('Dans les données de test, il y a {} transactions
frauduleuses'.format(sum(y_test)))
```

Les deux dernières instructions permettent d'afficher à titre indicatif le nombre de transactions détectées par le modèle en tant que transactions frauduleuses et le nombre de transactions qui sont réellement frauduleuses présentes dans les données de test. Ce modèle donne les résultats suivants :

```
Precision: 0.975 / Recall: 0.762 / Accuracy: 1.0
Le modèle prédit 79 transactions frauduleuses
Dans les données de test, il y a 101 transactions frauduleuses
```

Ces résultats montrent un niveau de la métrique `recall` intéressant puisqu'il est de 76.2 %, mais surtout ces résultats sont encore plus intéressants que ceux que nous avons obtenus avec tous les autres exemples précédents de ce chapitre puisque cette fois la métrique `precision` est quasiment parfaite !

6. Conclusion

L'algorithme SVM est l'un des algorithmes les plus sophistiqués parmi les algorithmes d'apprentissage automatique. Les deux premières sections ont abordé les SVM par des illustrations géométriques pour fournir au lecteur une compréhension intuitive de ces algorithmes SVM. La troisième section a introduit les notions mathématiques pour expliquer le problème d'optimisation sous-jacent aux SVM. La quatrième section a abordé les aspects théoriques relatifs à la fonction noyau utilisée dans le cadre des SVM pour traiter les cas où les données ne sont pas linéairement séparables. Nous avons terminé ce chapitre avec des exemples permettant de montrer d'une part comment appliquer les SVM sur un problème en utilisant Scikit-learn, et d'autre part comment manipuler les données et régler les paramètres des SVM afin d'améliorer les performances des modèles.